

RETAIL INVENTORY AND SALES MANAGEMENT SYSTEM

DBMS COURSE PROJECT REPORT

Student Name	Krish Chhotoo Ghadge
Roll Number	25WU0102132
Course	Database Management Systems (DBMS)
Project Type	Individual
Academic Year	2026-2027
Project Serial No.	8
Faculty	_____

PROJECT SUMMARY

The Retail Inventory and Sales Management System is a relational database application designed to manage retail products, suppliers, customers, orders, order details, customer reviews, and inventory adjustments. A MySQL database stores the persistent data while a Flask-based web application provides the user interface and executes parameterized SQL operations.

The project demonstrates the complete database-management workflow: conceptual modeling using an ER diagram, conversion to relational tables, normalization, SQL DDL/DML, database connectivity, user-interface operations, testing, and repository-based submission.

GitHub repository: DBMS-Course-Project
Report submission folder: Project-Report

2. ABSTRACT

Retail organizations handle several categories of related information simultaneously. Product records contain prices and stock quantities, suppliers provide products, customers place orders, individual products participate in orders, customers submit reviews, and stock movements need to be tracked. If these records are maintained independently, duplicate data and inconsistent updates can occur.

This project addresses the problem by organizing retail information into a connected relational database. The system uses MySQL for persistent storage and Python Flask as the application layer. The web interface provides structured pages for viewing and managing records while SQL statements perform the actual database operations.

3. INTRODUCTION

The system is designed around the idea that a retail operation can be represented as a set of entities and relationships. Customers place orders. An order can contain multiple products, and a product can occur in many orders; therefore, the Order Details relationship records quantity and unit price for each product in an order. Suppliers provide products. Customers can write reviews for products, while the inventory log records adjustments associated with products.

4. PROBLEM STATEMENT

Develop a database management system for a retail environment that stores related business records in a structured relational schema and provides a connected user interface for inserting, deleting, updating, and viewing records. The system should preserve relationships between entities and make common retail information accessible through SQL queries and the web UI.

The final-project instructions specifically require a MySQL-connected UI demonstrating insertion, deletion, and viewing of records with the corresponding database change shown during the demonstration.

5. OBJECTIVES

- Design an ER model representing the retail domain.
- Convert the model into relational tables with keys and constraints.
- Reduce redundancy through normalization.
- Implement SQL DDL and DML operations.
- Connect the web application to MySQL.
- Provide usable screens for database interaction.
- Test common database operations and relationship-sensitive cases.

6. SYSTEM SCOPE AND MODULES

Module	Main responsibility	Important data
Product Management	Maintain products, pricing and stock	ProductID, ProductName, Price, CostPrice, StockQuantity, ReorderLevel
Supplier Management	Maintain supplier information	supplierID, suppliername, contactname, phone, email
Customer Management	Maintain customer master records	customerID, name, dateofbirth, address
Order Management	View and manage customer orders	OrderID, CustomerID, OrderDate, TotalAmount, PaymentStatus
Order Details	Represent products within orders	OrderDetailID, OrderID, ProductID, Quantity, UnitPrice, LineTotal
Review Management	Store customer product feedback	ReviewID, CustomerID, ProductID, Rating, Comment
Inventory Logging	Record stock adjustments	LogID, ProductID, Adjustment, Reason, LogDate

7. SOFTWARE AND HARDWARE REQUIREMENTS

Layer / Tool	Technology
Operating system	Windows 10/11 or equivalent
Database	MySQL / MySQL Workbench
Backend	Python 3.x, Flask
Database connector	mysql-connector-python
Frontend	HTML5, CSS3, JavaScript
IDE	Visual Studio Code
Browser	Chrome / Edge / Firefox
Hardware	Standard laptop/desktop; 4 GB+ RAM recommended

8. SYSTEM ARCHITECTURE

Presentation layer: browser-rendered HTML templates, forms, navigation, tables, search controls and visual feedback.

Application layer: Flask routes receive requests, read form values, execute SQL, commit writes and render pages.

Database layer: MySQL stores the seven relations and enforces relational integrity.

The application flow is: User → Browser → Flask Route → Parameterized SQL → MySQL → Commit/Query Result → Updated UI.

9. ENTITY-RELATIONSHIP DIAGRAM

The following is the project ER diagram supplied for the database. It shows the entities, attributes, relationship names and cardinalities used to represent the retail domain.

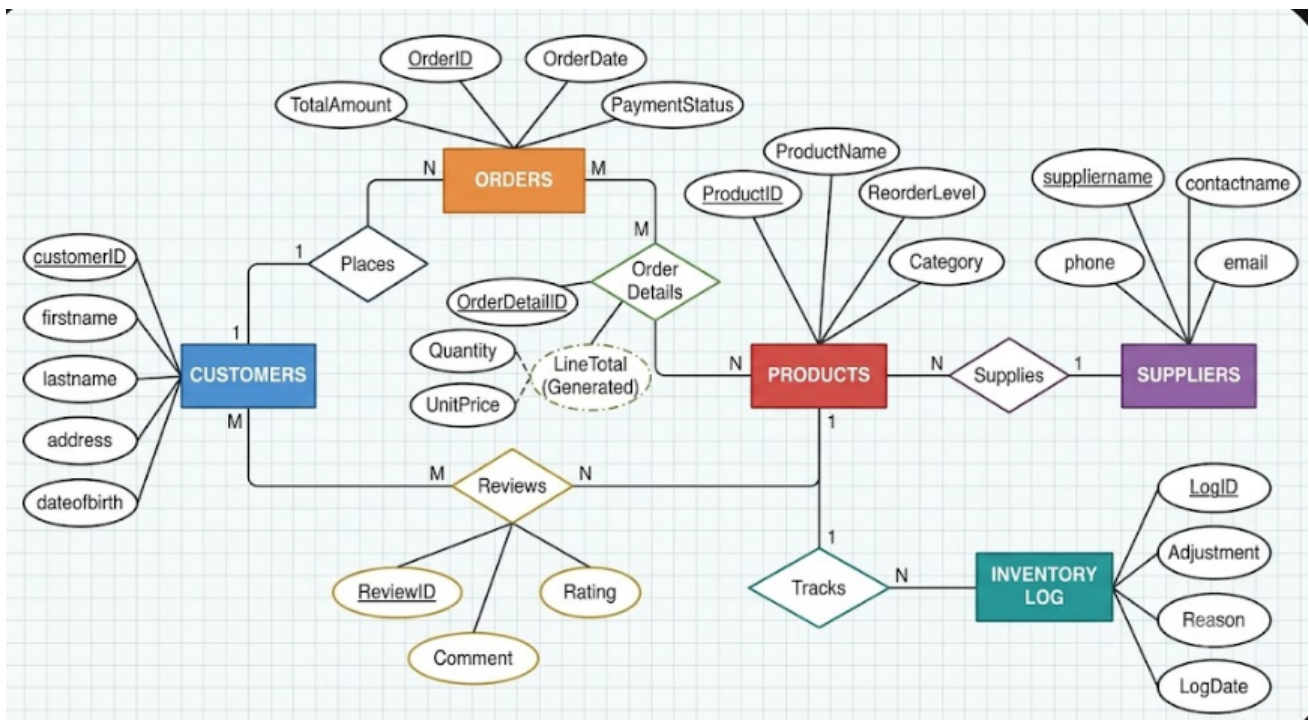


Figure 1. Project ER diagram. Underlined attributes represent key attributes; relationship labels and 1/M/N markers describe how records are associated.

10. ER DIAGRAM INTERPRETATION

CUSTOMERS — Places — ORDERS: One customer can place many orders, while each order is associated with one customer. This is represented as a 1:N relationship.

ORDERS — Order Details — PRODUCTS: Orders and products have a many-to-many business relationship. The Order Details relationship records OrderDetailID, Quantity, UnitPrice and the generated LineTotal, allowing each order to contain multiple products and each product to appear in multiple orders.

PRODUCTS — Supplies — SUPPLIERS: A supplier can supply multiple products, while a product is associated with a supplier. The diagram represents this as N:1 from products to suppliers.

CUSTOMERS — Reviews — PRODUCTS: Customers can review products. The review relationship carries ReviewID, Rating and Comment, allowing feedback to be associated with both the customer and product.

PRODUCTS — Tracks — INVENTORY LOG: A product can have multiple inventory-log entries. Each log entry records an adjustment, reason and date.

11. RELATIONAL SCHEMA

Table	Primary Key	Foreign Key(s)	Role
customers	customerID	—	Customer master entity
products	ProductID	supplierID → suppliers	Product and stock master
suppliers	supplierID	—	Supplier master entity
orders	OrderID	CustomerID → customers	Order header / transaction
orderdetails	OrderDetailID	OrderID → orders; ProductID → products	Order-product association
reviews	ReviewID	CustomerID → customers; ProductID → products	Product feedback
inventorylog	LogID	ProductID → products	Inventory adjustment history

12. DATA DICTIONARY

Table	Important attributes	Description
customers	firstname, lastname, dateofbirth, address	Describes the customer.
products	ProductName, Category, Description, Price, CostPrice, StockQuantity, ReorderLevel, LastStockDate	Describes products, its prices and stock.
suppliers	suppliername, contactname, address, phone, email, lastdeliverydate	Stores supplier contact and delivery information.
orders	OrderDate, TotalAmount, PaymentStatus, PaymentMethod, TrackingNumber	Stores order-level transaction information.
orderdetails	Quantity, UnitPrice, LineTotal	Stores the product-level part of an order.
reviews	Rating, Comment, ReviewDate	Stores customer feedback.
inventorylog	Adjustment, Reason, LogDate	Stores changes made to inventory.

13. NORMALIZATION AND INTEGRITY

The schema separates different real-world concepts into different relations. Customer information is stored in customers instead of being repeated in every order. Supplier information is stored separately from products. Orders are separated from order details because an order can contain multiple products. Reviews and inventory movements are also maintained independently.

First Normal Form (1NF): attributes represent individual values rather than repeating groups. For example, each order-detail record represents one product occurrence in an order.

Second Normal Form (2NF): attributes describe the relation in which they belong. Product attributes are stored with products, while quantity and unit price are stored with order details rather than with the order header.

Third Normal Form (3NF): non-key information is separated into the entity to which it belongs. Supplier contact information is not repeated for every product; customer details are not repeated in every order.

Entity integrity: primary keys uniquely identify records such as ProductID, customerID, OrderID and supplierID.

Referential integrity: foreign keys connect dependent records to existing parent records. This prevents relationships from pointing to nonexistent entities.

14. BUSINESS RULES

Rule	Explanation
Customer-Order	A customer may place multiple orders; an order belongs to a customer.
Supplier-Product	A supplier may supply multiple products.
Order-Product	An order can contain multiple products and a product can occur in multiple orders; Order Details resolves th
Customer-Review-Product	A review connects a customer with a product and stores rating/comment information.
Product-Inventory Log	A product can have multiple inventory adjustments over time.
Stock control	StockQuantity and ReorderLevel support identification of products requiring replenishment.

15. SQL COMMANDS AND QUERIES

DDL example:

```
CREATE TABLE products (ProductID INT PRIMARY KEY, ProductName VARCHAR(255), Category VARCHAR(100), Description TEXT, Price DECIMAL(10,2), CostPrice DECIMAL(10,2), supplierID INT, StockQuantity INT, ReorderLevel INT, LastRestockDate DATE);
```

INSERT:

```
INSERT INTO products (ProductName, Category, Description, Price, CostPrice, supplierID, StockQuantity, ReorderLevel, LastRestockDate) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s);
```

UPDATE:

```
UPDATE products SET ProductName=%s, Category=%s, Description=%s, Price=%s, CostPrice=%s, supplierID=%s, StockQuantity=%s, ReorderLevel=%s, LastRestockDate=%s WHERE ProductID=%s;
```

DELETE:

```
DELETE FROM products WHERE ProductID=%s;
```

Useful SELECT / JOIN queries:

Query	Purpose
SELECT * FROM products;	Display all product records.
SELECT * FROM customers;	Display all customers.
SELECT p.ProductName, s.suppliername FROM products p JOIN suppliers s ON p.supplierID = s.supplierID;	Display product and supplier information.
SELECT o.OrderID, c.firstname, c.lastname, o.TotalAmount FROM orders o JOIN customers c ON o.customerID = c.customerID;	Display orders with customer details.
SELECT od.OrderID, p.ProductName, od.Quantity, od.UnitPrice FROM orderdetails od JOIN products p ON od.productID = p.productID;	Display order details with product information.
SELECT ProductID, SUM(Adjustment) FROM inventorylog GROUP BY ProductID;	Summarize inventory adjustments.

Parameterized SQL is used in the Flask layer so values supplied by forms are passed as parameters instead of being directly concatenated into SQL strings.

16. SAMPLE DATABASE OUTPUTS

The product relation inspected during implementation contained representative records such as the following:

ProductID	ProductName	Category	Price	CostPrice	StockQuantity	ReorderLevel
101	Quantum X1 Smartphone	Electronics	899.99	550.00	110	15
102	Ergonomic Office Chair	Furniture	249.50	120.00	43	10
103	UltraClean Air Purifier	Appliances	189.99	95.00	59	12
104	Eco-Friendly Shipping Boxes	Packaging	35.00	15.00	300	50
105	Braided USB-C Cable (6ft)	Electronics	15.99	4.50	500	100

17. USER INTERFACE DESIGN

The UI is organized around a dashboard and table-based management screens. Navigation gives direct access to Products, Customers, Suppliers, Orders, Reviews and Inventory Log. The design emphasizes clear forms, searchable tables, status information and direct database actions.

The product-management workflow provides the clearest demonstration of CRUD behavior. A user can open the Products page, add a record, edit an existing record, and delete a record. After each write operation the page is refreshed from MySQL, allowing the user to see the changed state.

The final project instructions require screenshots for insert, delete, view and before/after states. The screenshots should therefore be included in the Presentation-III folder as well as referenced in the report.

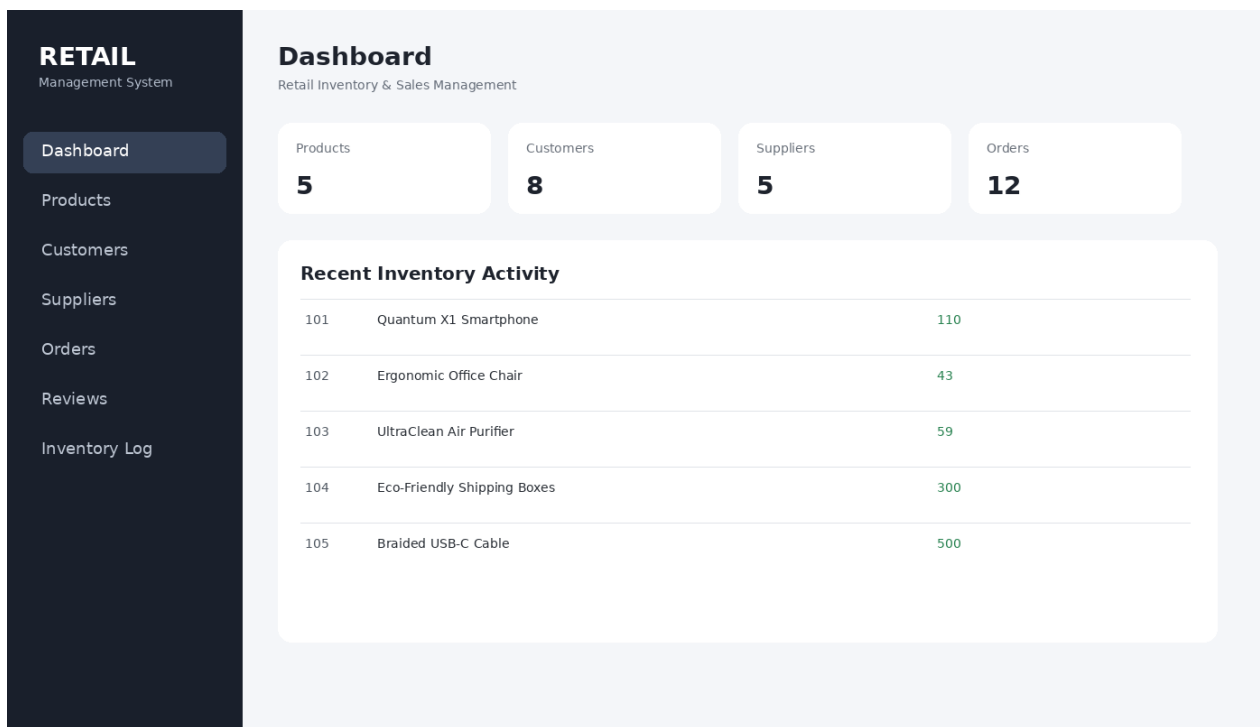


Figure 2. Dashboard interface used to access the major modules.

18. UI OPERATION SCREENS

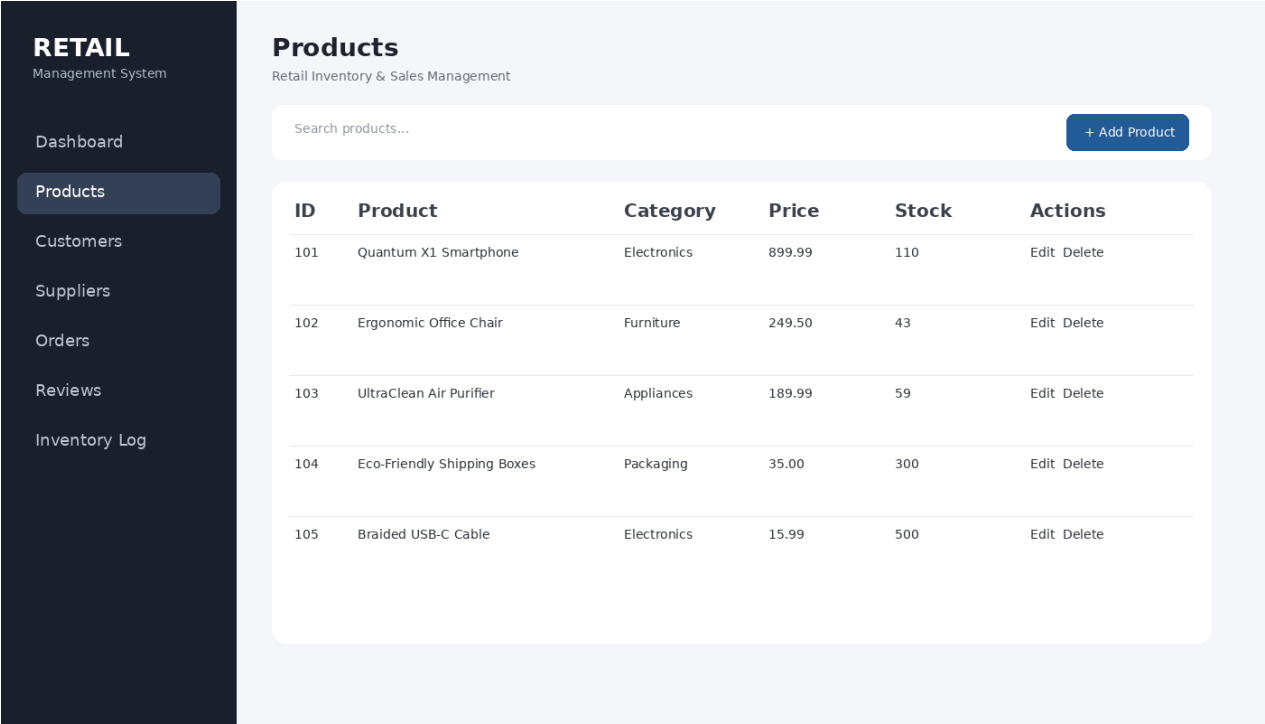


Figure 3. Product list and management interface.

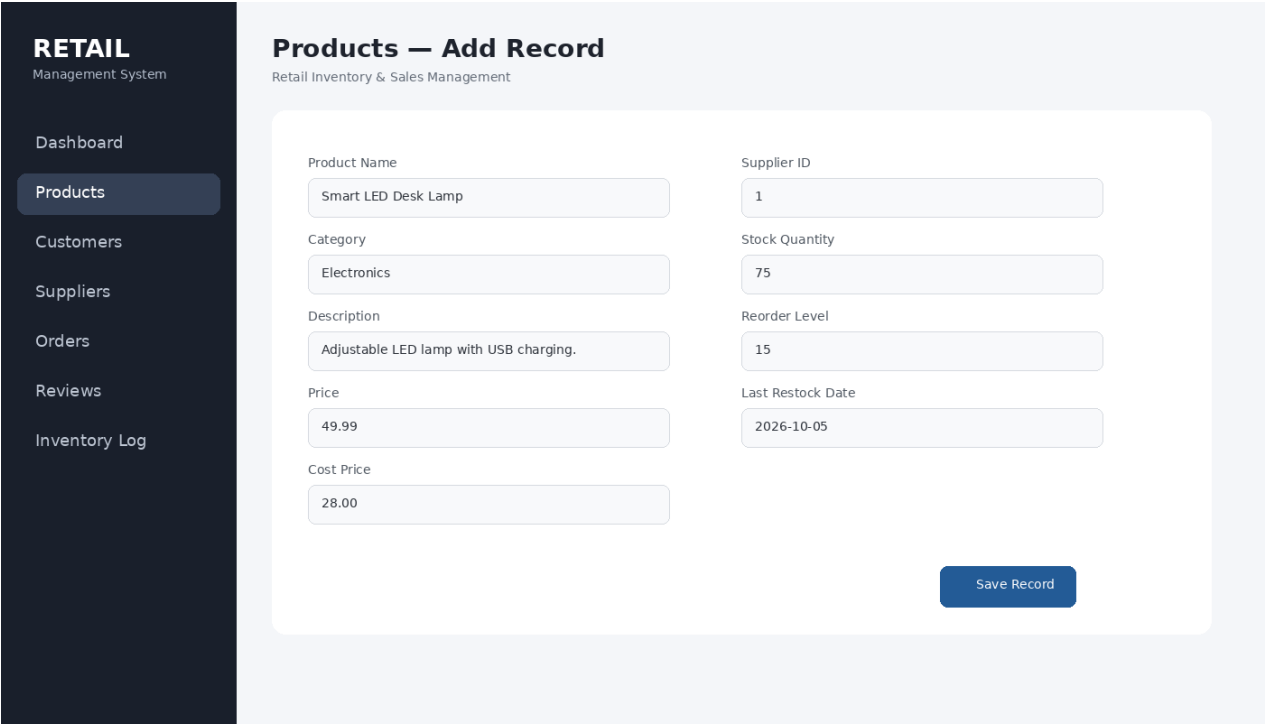


Figure 4. Product insertion interface.

19. IMPLEMENTATION, TESTING AND CONCLUSION

Implementation: Flask routes handle requests from the browser. The database connection is established using mysql-connector-python and the retailmanagement database. Form values are supplied to parameterized SQL statements. Successful INSERT, UPDATE and DELETE operations are committed to MySQL.

Testing approach: Each core UI operation is tested by performing the action in the browser and then checking the corresponding table through MySQL. This verifies that the interface is not merely changing HTML state but is actually modifying persistent database records.

Test Case	Procedure	Expected outcome
TC01 – View	Open Products page	Existing product records are retrieved from MySQL.
TC02 – Insert	Submit a valid product form	A new product record is stored and displayed.
TC03 – Update	Edit product price/stock/name	Updated values are persisted and displayed.
TC04 – Delete	Delete a selected product	The record is removed when no dependent constraint prevents deletion.
TC05 – Customer view	Open Customers page	Customer records are displayed.
TC06 – Supplier view	Open Suppliers page	Supplier records are displayed.
TC07 – Order view	Open Orders page	Order records and transaction fields are displayed.
TC08 – Relationship check	Query products with suppliers/orders	JOIN results show connected records.

20. CONCLUSION

The Retail Inventory and Sales Management System provides a practical example of database design applied to a retail scenario. The ER model identifies the core entities and relationships, while the relational schema separates those entities into manageable tables. Keys preserve identity and relationships, normalization reduces redundancy, and SQL provides the mechanism for storing and retrieving data.

The Flask interface adds an accessible application layer over MySQL. Through the UI, users can view records and perform database operations while the underlying tables remain the authoritative source of persistent information.

21. FUTURE ENHANCEMENTS

Potential improvements include role-based authentication, automatic stock deduction after order confirmation, low-stock notifications based on ReorderLevel, sales analytics, export to CSV/PDF, stronger form validation, audit trails, automated backups, and deployment using a production web server.

22. REFERENCES AND APPENDIX

1. MySQL / MySQL Workbench documentation.
 2. Flask documentation.
 3. Python and mysql-connector-python documentation.
 4. DBMS Course Project – Final Presentation Instructions, AIML Whales and AIML Panthers, 2026.
- GitHub: <https://github.com/<your-username>/DBMS-Course-Project>

Submission checklist: Presentation-I, Presentation-II, Presentation-III, Project-Report, source code, README.md, ER diagram, SQL file, UI screenshots and required query solution should be placed in the public GitHub repository as specified by the course instructions.